LIVE QA AUTOMATION REPORT

Static Code Analysis + Runtime Verification

Project: D:\ecommerce-store

Generated: 2026-07-17 11:29:53

Prepared by: Senior QA Automation Engineer

173	31+	16
JS/JSX files	page surfaces	verification steps

1. Project Information

This report combines static source-code inspection with live environment verification. The audited application is a React/Vite ecommerce storefront and admin dashboard with FastAPI backend, Docker Compose runtime, Redis, Nginx, and Supabase PostgreSQL connectivity.

Frontend stack observed: React 18, Vite, Redux Toolkit, React Query, Tailwind CSS, Firebase, Razorpay, lucide-react, react-hot-toast, and Playwright dependency inside the frontend container.

Routes reviewed include storefront home, products, product detail, custom products, offers, cart, wishlist, checkout, tracking, auth, profile, orders, support/footer pages, not-found, and admin dashboard/products/categories/custom products/orders/offers/customers/contact/settings/banners.

2. Environment Verification

Step	Command	Status	Summary
1	node -v	FAIL	node command not recognized.
2	npm -v	FAIL	npm command not recognized.
3	npm install	FAIL	Cannot execute because npm is not recognized.
4	npm run lint	FAIL	Cannot execute because npm is not recognized.
5	npm run build	FAIL	Cannot execute because npm is not recognized.
6	npm test	FAIL	Cannot execute because npm is not recognized.
7	docker --version	PASS	Docker CLI available.
8	docker compose version	PASS	Docker Compose available.
9a	docker compose build	FAIL	Sandbox attempt failed on Docker buildx permission.
9b	docker compose build (escalated retry)	PASS	Backend and frontend images built.
10a	docker compose up --detach	FAIL	Sandbox attempt could not connect to Docker API.
10b	docker compose up --detach (escalated retry)	PASS	Stack started; backend became healthy; frontend started.
11	Invoke-WebRequest http://localhost:5173	PASS	Frontend returned 200 OK.
12	Invoke-WebRequest http://localhost:8000/health	PASS	Backend health returned 200 OK.
13	DB connectivity via backend startup/logs	PASS	PostgreSQL ready, Alembic complete, seed complete.
14	Browser console capture	FAIL	Attempted through Playwright; Chromium could not launch in container.
15	Build errors	FAIL	Host npm build failed; Docker image build passed.
16	Runtime errors	PASS	Frontend/backend server logs did not show fatal runtime errors; one expected

3. Build Verification

Host NPM Build

Host build verification failed because node and npm are not recognized in this PowerShell environment. This is proven by direct command output from node -v, npm -v, npm install, npm run lint, npm run build, and npm test.

Docker Build

Docker Compose build initially failed inside sandbox due Docker config/buildx access. Escalated retry passed and produced backend and frontend images successfully.

Test Verification

npm test was attempted in frontend and failed because npm is not recognized on the host. No test suite result could be produced from host npm. Docker container dependency startup reported frontend dependencies up-to-date, but no dedicated test command was executed inside container because the requested command was npm test.

4. Runtime Verification

Docker Compose up was executed in detached mode to avoid leaving a foreground process running. The sandbox attempt failed due Docker API permission, then escalated retry passed.

Container status showed ecommerce-store-frontend-1 up on port 5173, ecommerce-store-backend-1 up and healthy on port 8000, nginx up on port 80, adminer up on 8080, and redis running.

Frontend verification: http://localhost:5173 returned 200 OK.

Backend verification: http://localhost:8000/health returned 200 OK with content {status: healthy, version: 1.0.0}.

Database connectivity: backend logs show PostgreSQL Ready, Alembic Complete, database seed complete, and live API endpoints returning 200. This verifies backend-to-database connectivity through application startup and API behavior.

Runtime logs: no fatal backend/frontend runtime errors were captured. Backend showed one GET /api/v1/auth/me 401, which is expected for an unauthenticated request.

5. Docker Verification

Docker CLI: PASS with Docker version 29.6.1, build 8900f1d. The CLI also emitted a config access warning.

Docker Compose: PASS with Docker Compose version v5.2.0. Compose build and up passed after escalation.

Important note: The non-escalated Docker API/buildx attempts failed due local Docker config/API permissions. The final Docker runtime verification passed outside sandbox.

6. UI Analysis

The UI has a solid structure and includes meaningful shells for storefront/admin, product grids, admin tables, checkout overlays, cart drawer, and reusable UI primitives. However, the visual system is inconsistent because several modules bypass theme tokens and use hard-coded black, white, gray, or hex colors.

The highest-impact UI fixes are dark-mode consistency, brand token repair, footer color cleanup, catalog title and toolbar token cleanup, contact inbox polish, and mobile product-card CTA visibility.

7. Component Analysis

Working Properly

- CartDrawer and CartEmpty have strong baseline states.
- ProductGrid has skeleton and empty state support.
- Checkout PaymentSection handles loading and no-payment-method states.
- Admin ProductsPage has mobile cards, desktop table, pagination, and error boundary.
- Docker runtime verified frontend and backend serving successfully.

Needs Changes

- StoreHeader slug routing.
- StoreFooter dark-mode tokens.
- HeroSection fallback styling.
- ProductList filter/search consistency.
- ProductCard mobile CTA and rating.
- FilterDrawer mobile width.
- ProductDetails and Checkout auth redirects.
- ContactMessagesPage mojibake and theme cleanup.
- Global console.log cleanup.

Unused / Duplicate / Suspicious

- ProductFilters imported but not rendered in ProductList.
- CustomerProfilePage appears not routed.
- Duplicate Modal and Badge components exist under shared/common and shared/ui.
- Duplicate cart drawer paths exist under storeheaders and shoppingcart.
- Admin and shared PageHeader variants create inconsistent page headers.

8. Issue List

1. Theme tokens: Tailwind brand scale is all black; restore semantic brand colors.
2. Dark mode: Footer, hero, catalog, admin contact, and dashboard use hard-coded light colors.
3. Navigation: Product details and checkout still use legacy /login instead of /auth/login.
4. Catalog: Mobile menu category slugs are plural and can mismatch backend canonical category slugs.
5. Mobile UX: ProductCard add-to-cart is hover-only; make it visible on touch devices.
6. Filters: FilterDrawer is fixed at 420px and can overflow narrow mobile screens.
7. Code quality: Production console.log statements remain in auth/product/order/admin flows.
8. Contact inbox: Mojibake symbols and raw light-mode UI reduce professional polish.
9. Forms: Auth and checkout need persistent inline errors with aria-describedby.
10. Performance: CSS @import for fonts and image dimension gaps can affect rendering and layout shift.

9. Recommendations

Critical Fixes

- Fix /login redirects to /auth/login.
- Normalize category slugs.
- Repair Tailwind brand tokens and theme variables.
- Make product add-to-cart visible on mobile.
- Fix FilterDrawer width with w-full max-w-[420px].

Medium Fixes

- Remove or guard production console.log statements.
- Replace mojibake with lucide icons/text.
- Add inline validation and aria-describedby.
- Standardize duplicate shared UI primitives.
- Move font loading out of CSS @import and add image dimensions where possible.

Minor Fixes

- Fix cart CTA typo.
- Convert nested absolute route paths to relative child paths.
- Improve category empty/error state.
- Add stronger screenshot-based regression coverage.

10. Final Score

Readiness: Moderate. Runtime Docker verification passed, but host npm tooling is unavailable, browser console capture is blocked by Chromium runtime compatibility in the container, and multiple P1/P2 UI issues remain.

UI Score 72 / 100	**UX Score** 68 / 100	**Accessibility** 64 / 100
Performance 70 / 100	**Responsive** 66 / 100	**Code Quality** 69 / 100

Overall Health

68%

11. Appendix - Command Outputs

Step 1 - node -v - FAIL

node : The term 'node' is not recognized as the name of a cmdlet, function, script file, or operable program.

Step 2 - npm -v - FAIL

npm : The term 'npm' is not recognized as the name of a cmdlet, function, script file, or operable program.

Step 3 - npm install - FAIL

npm : The term 'npm' is not recognized as the name of a cmdlet, function, script file, or operable program.

Step 4 - npm run lint - FAIL

npm : The term 'npm' is not recognized as the name of a cmdlet, function, script file, or operable program.

Step 5 - npm run build - FAIL

npm : The term 'npm' is not recognized as the name of a cmdlet, function, script file, or operable program.

Step 6 - npm test - FAIL

npm : The term 'npm' is not recognized as the name of a cmdlet, function, script file, or operable program.

Step 7 - docker --version - PASS

WARNING: Error loading config file: open C:\Users\ELCOT\.docker\config.json: Access is denied.

Docker version 29.6.1, build 8900f1d

Step 8 - docker compose version - PASS

WARNING: Error loading config file: open C:\Users\ELCOT\.docker\config.json: Access is denied.

Docker Compose version v5.2.0

Step 9a - docker compose build - FAIL

CreateFile C:\Users\ELCOT\.docker\buildx\instances: Access is denied.

Step 9b - docker compose build (escalated retry) - PASS

Image ecommerce-store-backend Built

Image ecommerce-store-frontend Built

Step 10a - docker compose up --detach - FAIL

unable to get image 'redis:7-alpine': permission denied while trying to connect to the docker API at npipe:////./pipe/docker_engine

Step 10b - docker compose up --detach (escalated retry) - PASS

Container ecommerce-store-backend-1 Healthy

Container ecommerce-store-frontend-1 Started

Step 11 - Invoke-WebRequest http://localhost:5173 - PASS

StatusCode 200 OK

Step 12 - Invoke-WebRequest http://localhost:8000/health - PASS

StatusCode 200 OK Content {"status":"healthy","version":"1.0.0"}

Step 13 - DB connectivity via backend startup/logs - PASS

[Startup] PostgreSQL Ready

[Startup] Alembic Complete

[Startup] Database Seed Complete

Step 14 - Browser console capture - FAIL

browserType.launch: Failed to launch: Error: spawn /root/.cache/ms-playwright/chromium-1228/chrome-linux64/chrome ENOENT

Step 15 - Build errors - FAIL

Host: npm command not recognized.

Docker: backend and frontend images built.

Step 16 - Runtime errors - PASS

frontend: VITE ready in 757 ms.

backend: Application startup complete.

backend: GET /api/v1/auth/me 401 (expected without auth).